



Tribhuvan University
Institute of Engineering
Purwanchal Campus
पूर्वाञ्चल क्याम्पस

Unit 3

Advanced Data Wrangling and Transformation

Contents

- 3.1 Advanced Pandas operations: Merging, joining, reshaping, pivoting
- 3.2 Handling missing, categorical, and time-series data
- 3.3 Feature transformation, scaling, and encoding
- 3.4 Memory optimization and efficient data processing
- 3.5 Building a reusable data-cleaning pipeline
- 3.6 Introduction to data pipeline components

Contents

- 3.1 Advanced Pandas operations: Merging, joining, reshaping, pivoting**
- 3.2 Handling missing, categorical, and time-series data
- 3.3 Feature transformation, scaling, and encoding
- 3.4 Memory optimization and efficient data processing
- 3.5 Building a reusable data-cleaning pipeline
- 3.6 Introduction to data pipeline components

Pandas

- Pandas is a Python library for data analysis. It is built on top of NumPy. It provides fast and flexible data structures.
- The two main data structures are:

Series (1D)

DataFrame (2D)



Pandas

- Pandas is widely used for:
 - 1) Data cleaning
 - 2) Data transformation
 - 3) Data analysis
 - 4) Data visualization support



Why is Pandas Important?

- Handles real-world data easily
- Works well with CSV, Excel, SQL, JSON
- Integrates with NumPy, Matplotlib, Scikit-learn



```
import pandas as pd
data = {
    'City': ['Kathmandu', 'Pokhara', 'Lalitpur'],
    'Population': [1000000, 500000, 300000]
}
df = pd.DataFrame(data)
print(df)
```

✓ 0.0s

	City	Population
0	Kathmandu	1000000
1	Pokhara	500000
2	Lalitpur	300000

Advanced Pandas Operations

- 1) Merging
- 2) Joining
- 3) Reshaping
- 4) Pivoting



Merging DataFrames

- Merging combines DataFrames using a common column.
- Similar to SQL JOIN.
- Types of Merge
 - 1) Inner
 - 2) Left
 - 3) Right
 - 4) Outer

Merging DataFrames – Inner Join

```
df_students = pd.DataFrame({  
    'StudentID': [1, 2, 3],  
    'Name': ['Ram', 'Sita', 'Hari']  
})
```

```
df_marks = pd.DataFrame({  
    'StudentID': [1, 2, 4],  
    'Marks': [85, 90, 75]  
})
```

Merging DataFrames – Inner Join

	StudentID	Name
0	1	Ram
1	2	Sita
2	3	Hari

	StudentID	Marks
0	1	85
1	2	90
2	4	75

```
merged_inner = pd.merge(df_students, df_marks, on='StudentID', how='inner')  
print(merged_inner)
```

	StudentID	Name	Marks
0	1	Ram	85
1	2	Sita	90

Merging DataFrames – Left Join

	StudentID	Name
0	1	Ram
1	2	Sita
2	3	Hari

	StudentID	Marks
0	1	85
1	2	90
2	4	75

```
merged_left = pd.merge(df_students, df_marks, on='StudentID', how='left')  
print(merged_left)
```

	StudentID	Name	Marks
0	1	Ram	85.0
1	2	Sita	90.0
2	3	Hari	NaN

Merging DataFrames – Right Join

	StudentID	Name
--	-----------	------

0	1	Ram
---	---	-----

1	2	Sita
---	---	------

2	3	Hari
---	---	------

	StudentID	Marks
--	-----------	-------

0	1	85
---	---	----

1	2	90
---	---	----

2	4	75
---	---	----

```
merged_right = pd.merge(df_students, df_marks, on='StudentID', how='right')  
print(merged_right)
```

	StudentID	Name	Marks
0	1	Ram	85
1	2	Sita	90
2	4	NaN	75

Merging DataFrames – Outer Join

	StudentID	Name
--	-----------	------

0	1	Ram
---	---	-----

1	2	Sita
---	---	------

2	3	Hari
---	---	------

	StudentID	Marks
--	-----------	-------

0	1	85
---	---	----

1	2	90
---	---	----

2	4	75
---	---	----

```
merged_outer = pd.merge(df_students, df_marks, on='StudentID', how='outer')
print(merged_outer)
```

	StudentID	Name	Marks
0	1	Ram	85.0
1	2	Sita	90.0
2	3	Hari	NaN
3	4	NaN	75.0

Joining DataFrames

- Join combines DataFrames using index instead of columns.

```
df_A = pd.DataFrame({  
    'Temperature': [20, 22, 24]  
}, index=['Day1', 'Day2', 'Day3'])
```

```
df_B = pd.DataFrame({  
    'Humidity': [60, 65, 70]  
}, index=['Day1', 'Day2', 'Day4'])
```

Joining DataFrames - Inner

Temperature

Day1	20
------	----

Day2	22
------	----

Day3	24
------	----

Humidity

Day1	60
------	----

Day2	65
------	----

Day4	70
------	----

```
joined_df = df_A.join(df_B, how='inner')  
print(joined_df)
```

	Temperature	Humidity
Day1	20	60
Day2	22	65

Joining DataFrames - Left

Temperature	
Day1	20
Day2	22
Day3	24

Humidity	
Day1	60
Day2	65
Day4	70

```
joined_left = df_A.join(df_B)  
print(joined_left)
```

	Temperature	Humidity
Day1	20	60.0
Day2	22	65.0
Day3	24	NaN

Joining DataFrames - Right

Temperature		Humidity	
Day1	20	Day1	60
Day2	22	Day2	65
Day3	24	Day4	70

```
outerjoined_df = df_A.join(df_B, how='outer')  
print(outerjoined_df)
```

	Temperature	Humidity
Day1	20.0	60.0
Day2	22.0	65.0
Day3	24.0	NaN
Day4	NaN	70.0

Reshaping Dataframes

- Reshaping means changing the structure of a DataFrame.
- The data values remain the same. Only the layout changes.
- Common reshaping operations:
 - melt
 - pivot
 - pivot_table
 - stack
 - unstack

Wide vs Long Data Format

Wide format:

- Each variable is a separate column
- Easy for humans to read

	Day	Temp	Humidity
0	Mon	20	60
1	Tue	22	65

```
df_wide = pd.DataFrame({  
    'Day': ['Mon', 'Tue'],  
    'Temp': [20, 22],  
    'Humidity': [60, 65]  
})  
print(df_wide)
```

Wide vs Long Data Format

Long format:

- One column for variable names
- One column for values
- Preferred for analysis and plotting

	Day	Variable	Value
0	Mon	Temp	20
1	Mon	Humidity	60
2	Tue	Temp	22
3	Tue	Humidity	65

```
df_long = pd.DataFrame({  
    'Day': ['Mon', 'Mon', 'Tue', 'Tue'],  
    'Variable': ['Temp', 'Humidity', 'Temp', 'Humidity'],  
    'Value': [20, 60, 22, 65]  
})  
print(df_long)
```

Reshaping Dataframes - Melt

- (Wide to Long)
- Melt converts columns into rows.

	Student	Math	Science
0	Ram	80	75
1	Sita	85	90

```
df_scores = pd.DataFrame({  
    'Student': ['Ram', 'Sita'],  
    'Math': [80, 85],  
    'Science': [75, 90]  
})  
print(df_scores)
```

Reshaping Dataframes - Melt

```
melted = pd.melt(  
df_scores,  
id_vars=['Student'],  
var_name='Subject',  
value_name='Score'  
)  
print(melted)
```

Key parameters:

id_vars: columns to keep fixed

var_name: name of new variable column

value_name: name of value column

	Student	Math	Science
0	Ram	80	75
1	Sita	85	90



	Student	Subject	Score
0	Ram	Math	80
1	Sita	Math	85
2	Ram	Science	75
3	Sita	Science	90

Reshaping Dataframes - Pivot

- (Long to Wide)
- Pivot reshapes data using unique values.
- No aggregation allowed.

	Student	Subject	Score
0	Ram	Math	80
1	Ram	Science	75
2	Sita	Math	85
3	Sita	Science	90

```
df_long = pd.DataFrame({  
    'Student': ['Ram', 'Ram', 'Sita', 'Sita'],  
    'Subject': ['Math', 'Science', 'Math', 'Science'],  
    'Score': [80, 75, 85, 90]  
})  
print(df_long)
```


Reshaping Dataframes - Pivot

```
pivoted = df_long.pivot(  
    index='Student',  
    columns='Subject',  
    values='Score'  
)  
print(pivoted)
```

	Student	Subject	Score
0	Ram	Math	80
1	Ram	Science	75
2	Sita	Math	85
3	Sita	Science	90



Subject	Math	Science
Student		
Ram	80	75
Sita	85	90

If duplicates exist, pivot will raise an error.

Reshaping Dataframes – Stack

- Stack compresses columns into a multi-level index.
- Result is a Series with MultiIndex.

	Student	Subject	Score
0	Ram	Math	80
1	Ram	Science	75
2	Sita	Math	85
3	Sita	Science	90

```
df_long = pd.DataFrame({  
    'Student': ['Ram', 'Ram', 'Sita', 'Sita'],  
    'Subject': ['Math', 'Science', 'Math', 'Science'],  
    'Score': [80, 75, 85, 90]  
})  
print(df_long)
```

Reshaping Dataframes – Stack

- Stack compresses columns into a multi-level index.
- Result is a Series with MultiIndex.

	Student	Subject	Score
0	Ram	Math	80
1	Ram	Science	75
2	Sita	Math	85
3	Sita	Science	90

```
df_long = pd.DataFrame({  
    'Student': ['Ram', 'Ram', 'Sita', 'Sita'],  
    'Subject': ['Math', 'Science', 'Math', 'Science'],  
    'Score': [80, 75, 85, 90]  
})  
print(df_long)
```

Reshaping Dataframes – Stack

```
stacked = df_long.stack()  
print(stacked)
```

	Student	Subject	Score
0	Ram	Math	80
1	Ram	Science	75
2	Sita	Math	85
3	Sita	Science	90



0	Student	Ram
	Subject	Math
	Score	80
1	Student	Ram
	Subject	Science
	Score	75
2	Student	Sita
	Subject	Math
	Score	85
3	Student	Sita
	Subject	Science
	Score	90

dtype: object

Reshaping Dataframes – Stack

0	Student	Ram
	Subject	Math
	Score	80
1	Student	Ram
	Subject	Science
	Score	75
2	Student	Sita
	Subject	Math
	Score	85
3	Student	Sita
	Subject	Science
	Score	90

dtype: object

- Unstack is the reverse of stack.

```
df_unstacked = stacked.unstack()  
print(df_unstacked)
```



	Student	Subject	Score
0	Ram	Math	80
1	Ram	Science	75
2	Sita	Math	85
3	Sita	Science	90

Pivoting Dataframes – Pivot Table

- Pivot table is an advanced version of pivot.
- It supports aggregation.
- Pivot table handles duplicates using aggregation.

	Student	Subject	Score
0	Ram	Math	80
1	Ram	Math	85
2	Sita	Science	90
3	Sita	Science	88

```
df_exam = pd.DataFrame({  
    'Student': ['Ram', 'Ram', 'Sita', 'Sita'],  
    'Subject': ['Math', 'Math', 'Science', 'Science'],  
    'Score': [80, 85, 90, 88]  
})  
print(df_exam)
```

Pivoting Dataframes – Pivot Table

```
pivot_tbl = pd.pivot_table(  
    df_exam,  
    index='Student',  
    columns='Subject',  
    values='Score',  
    aggfunc='mean')  
print(pivot_tbl)
```

Student	Subject	Score
Ram	Math	80
Ram	Math	85
Sita	Science	90
Sita	Science	88



Subject	Math	Science
Student		
Ram	82.5	NaN
Sita	NaN	89.0

Common aggfunc values:

mean, sum, min, max, count

Contents

3.1 Advanced Pandas operations: Merging, joining, reshaping, pivoting

3.2 Handling missing, categorical, and time-series data

3.3 Feature transformation, scaling, and encoding

3.4 Memory optimization and efficient data processing

3.5 Building a reusable data-cleaning pipeline

3.6 Introduction to data pipeline components

Handling Real-World Data

- Real-world data is messy.
- Missing values, categories, and time information must be handled carefully.

Handling Missing Data

- What is Missing Data?
 - Missing data appears as: NaN, None, NaT (for time-series)
- Common causes:
 - Sensor failure
 - Manual data entry errors
 - Data collection issues

Handling Missing Data

ID	Age	Department	Salary	JoinDate
1	25	HR	45000	2022-01-10
2		IT	60000	2021-11-05
3	32	Finance		2020-06-18
4	28		52000	
5	45	HR	70000	2019-03-22
6	30	IT		2022-08-01
7		Marketing	48000	2021-02-14
8	38	Finance	65000	
9	26		50000	2023-01-09
10	41	Marketing		2020-12-30

Handling Missing Data

```
df = pd.read_csv("data.csv")  
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 10 entries, 0 to 9
```

```
Data columns (total 5 columns):
```

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	ID	10 non-null	int64
1	Age	8 non-null	float64
2	Department	8 non-null	object
3	Salary	7 non-null	float64
4	JoinDate	8 non-null	object

```
dtypes: float64(2), int64(1), object(2)
```

```
memory usage: 532.0+ bytes
```

```
None
```

Handling Missing Data

```
print(df.isna().sum())
```

ID	0
Age	2
Department	2
Salary	3
JoinDate	2
dtype:	int64

Handling Missing Data

```
# Drop rows with any missing value  
df_drop = df.dropna()  
print(df_drop)
```

```
# Drop columns with missing values  
df_drop_col = df.dropna(axis=1)  
print(df_drop_col)
```

Handling Missing Data

```
# Fill with constant  
df_filled = df.fillna(0)  
print(df_filled)
```

```
# Fill with mean  
df_mean = df.fillna(df.mean())  
print(df_mean)
```

Handling Missing Data

```
# Forward fill  
df_ffill = df.fillna(method='ffill')  
print(df_ffill)
```

```
# Backward fill  
df_bfill = df.fillna(method='bfill')  
print(df_bfill)
```


Handling Categorical Data

- Categorical data represents labels or groups. It is not numeric by nature.
- Examples:
 - Department
 - Gender
 - Product type
 - Education level
- Categorical data must be handled properly before analysis or ML.

Identifying Categorical Data

```
import pandas as pd

df = pd.DataFrame({
    'Department': ['HR', 'IT', 'Finance', 'HR'],
    'Level': ['Junior', 'Senior', 'Junior', 'Mid']
})
print(df.dtypes)
```

✓ 0.0s

Department	object
Level	object
dtype:	object

Converting to Category Data Type

```
df['Department'] = df['Department'].astype('category')  
df['Level'] = df['Level'].astype('category')  
  
print(df.dtypes)
```

✓ 0.0s

Department	category
Level	category
dtype:	object

Why use category type?:

- uses less memory
- faster comparisons
- clearer intent

Viewing Categories

```
df['Level'] = df['Level'].cat.rename_categories({  
    'Junior': 'Jr',  
    'Senior': 'Sr'  
})  
  
print(df)
```

	Department	Level
0	HR	Jr
1	IT	Sr
2	Finance	Jr
3	HR	Mid

Category Codes

```
df['Dept_Code'] = df['Department'].cat.codes  
print(df)
```

✓ 0.0s

	Department	Level	Dept_Code
0	HR	Jr	1
1	IT	Sr	2
2	Finance	Jr	0
3	HR	Mid	1

Handling Time-Series Data

- Time-series data is indexed by time. Each row represents an observation at a specific moment.
- Common examples:
 - daily sales
 - hourly website traffic
 - monthly revenue
- Handling time-series correctly is critical for analysis and forecasting.

```
import pandas as pd

df = pd.DataFrame({
    'Date': ['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04'],
    'Sales': [120, 135, 128, 140]
})
print(df)
```

✓ 0.0s

	Date	Sales
0	2024-01-01	120
1	2024-01-02	135
2	2024-01-03	128
3	2024-01-04	140

```
import pandas as pd

df = pd.read_csv(
    "data.csv",
    parse_dates=["date"],
    index_col="date"
)

print(df.head())
```

parse_dates converts strings to datetime.

index_col sets time as index.

Converting to Datetime

```
df['Date'] = pd.to_datetime(df['Date'])  
print(df.dtypes)
```

✓ 0.0s

```
Date      datetime64[ns]  
Sales      int64  
dtype: object
```

Setting Datetime Index

```
df = df.set_index('Date')  
print(df)
```

✓ 0.0s

	Sales
Date	
2024-01-01	120
2024-01-02	135
2024-01-03	128
2024-01-04	140

Extract Parts of Time

```
df["year"] = df.index.year  
df["month"] = df.index.month  
df["day"] = df.index.day  
df["weekday"] = df.index.day_name()  
print(df)
```

✓ 0.0s

	Sales	year	month	day	weekday
Date					
2024-01-01	120	2024	1	1	Monday
2024-01-02	135	2024	1	2	Tuesday
2024-01-03	128	2024	1	3	Wednesday
2024-01-04	140	2024	1	4	Thursday

Selecting Time Ranges

```
df.loc["2024-01-01":"2024-01-02"]
```

✓ 0.0s

	Sales	year	month	day	weekday
Date					
2024-01-01	120	2024	1	1	Monday
2024-01-02	135	2024	1	2	Tuesday

Selecting Time Ranges

```
df.loc["2024"]
```

✓ 0.0s

	Sales	year	month	day	weekday
Date					
2024-01-01	120	2024	1	1	Monday
2024-01-02	135	2024	1	2	Tuesday
2024-01-03	128	2024	1	3	Wednesday
2024-01-04	140	2024	1	4	Thursday

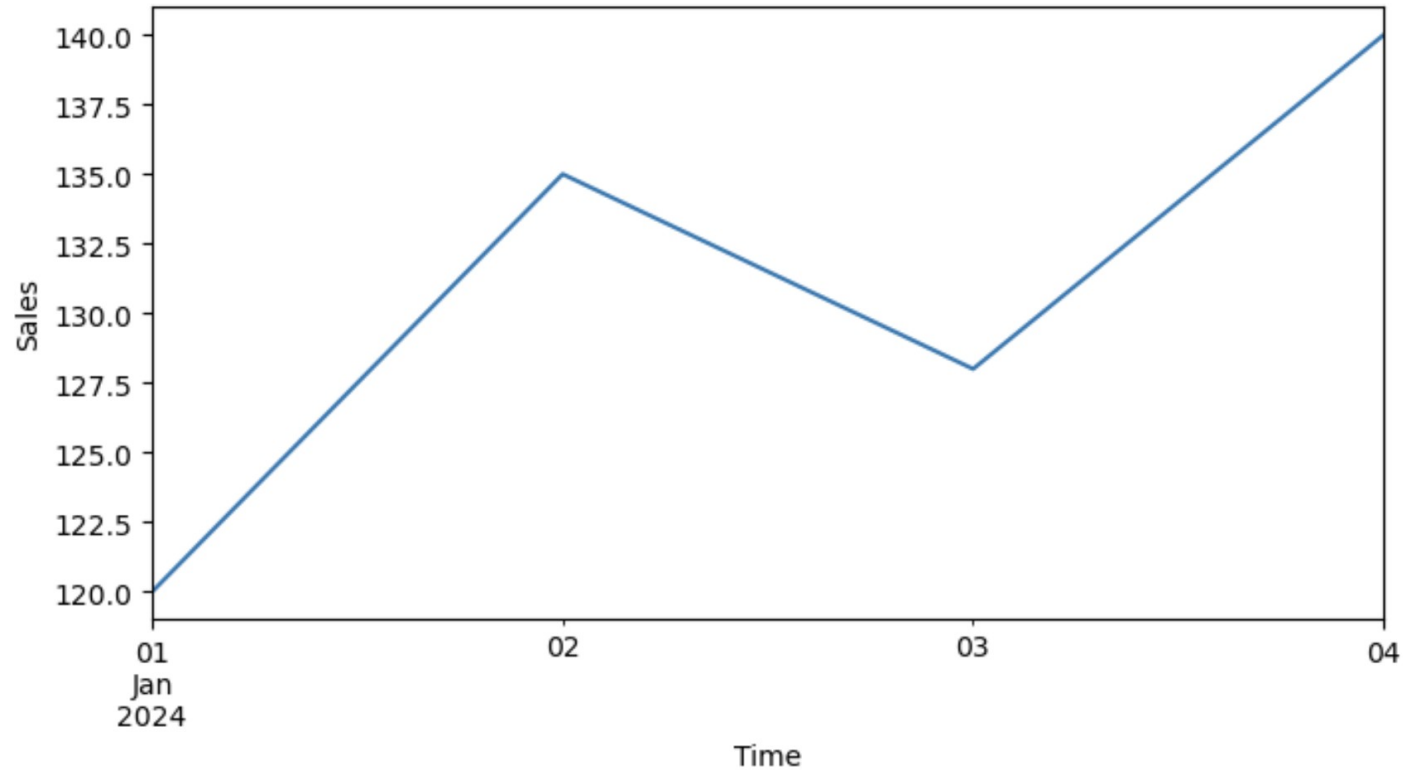
```
import matplotlib.pyplot as plt
```

```
df["Sales"].plot(figsize=(8, 4))
```

```
plt.xlabel("Time")
```

```
plt.ylabel("Sales")
```

```
plt.show()
```



Contents

- 3.1 Advanced Pandas operations: Merging, joining, reshaping, pivoting
- 3.2 Handling missing, categorical, and time-series data
- 3.3 Feature transformation, scaling, and encoding**
- 3.4 Memory optimization and efficient data processing
- 3.5 Building a reusable data-cleaning pipeline
- 3.6 Introduction to data pipeline components

Feature Transformation, Scaling, and Encoding

- Machine learning models work with numbers.
- Raw data is often messy.
- Features can be on different scales, have skewed distributions, or be categorical.
- Feature transformation fixes this.
- Goals:
 - Make features comparable
 - Improve model performance
 - Speed up training
 - Help models converge

Feature Transformation

- Feature transformation changes the shape or distribution of data.
- Common problems in raw features
 - Skewed data (e.g., income, population)
 - Outliers
 - Nonlinear relationships

Log Transformation

- Used when data is right-skewed.
- Formula: $\log(x + 1)$

```
import numpy as np
import pandas as pd
```

```
x = pd.Series([1, 10, 100, 1000])
x_log = np.log1p(x)
print(x_log)
```

✓ 0.0s

```
0    0.693147
1    2.397895
2    4.615121
3    6.908755
dtype: float64
```

Square Root Transformation

- Reduces moderate skewness.
- Formula: $\sqrt{x}$

```
import numpy as np
import pandas as pd
```

```
x = pd.Series([1, 10, 100, 1000])
x_sqrt = np.sqrt(x)
print(x_sqrt)
```

✓ 0.0s

```
0    1.000000
1    3.162278
2   10.000000
3   31.622777
dtype: float64
```

Power Transformation

- Used when distribution is unknown.
 - Box-Cox: only positive values
 - Yeo-Johnson: allows zero and negative values

```
from sklearn.preprocessing import PowerTransformer
```

```
pt = PowerTransformer(method='yeo-johnson')  
x_transformed = pt.fit_transform(x.values.reshape(-1, 1))  
print(x_transformed)
```

✓ 2.4s

```
[[-1.30817404]  
 [-0.50141245]  
 [ 0.45759942]  
 [ 1.35198707]]
```

Normalization

- **Min-max normalization** performs a linear transformation on the original data.
- Suppose that $\min_A$ and $\max_A$ are the minimum and maximum values of an attribute, A .
- Min-max normalization maps a value, v_i , of A to v_i' in the range $[\text{new_min}_A, \text{new_max}_A]$ by computing

$$v_i' = \frac{v_i - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A$$

- Suppose that the minimum and maximum values for the attribute income are \$12,000 and \$98,000, respectively. We would like to map income to the range $[0.0, 1.0]$. By min-max normalization, a value of \$73,600 for income is transformed to

$$\frac{73,600 - 12,000}{98,000 - 12,000} (1.0 - 0) + 0 = 0.716$$

Normalization

```
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# Example data
data = np.array([[200], [300], [400], [600], [1000]])
# Initialize the scaler
scaler = MinMaxScaler(feature_range=(0,1))
# Fit and transform the data
scaled_data = scaler.fit_transform(data)
print("Original data:\n", data)
print("Scaled data:\n", scaled_data)
```

Original data:

```
[[ 200]
 [ 300]
 [ 400]
 [ 600]
[1000]]
```

Scaled data:

```
[[0.   ]
 [0.125]
 [0.25 ]
 [0.5  ]
 [1.   ]]
```

Normalization

- In **z-score normalization** (or zero-mean normalization), the values for an attribute, **A**, are normalized based on the mean (i.e., average) and standard deviation of A.
- A value, **v_i** , of **A** is normalized to **v_i'** by computing

$$v_i' = \frac{v_i - \bar{A}}{\sigma_A}$$

- Suppose that the mean and standard deviation of the values for the attribute income are **\$54,000** and **\$16,000**, respectively. With z-score normalization, a value of **\$73,600** for income is transformed to

$$\frac{73,600 - 54,000}{16,000} = 1.225$$

Normalization

```
from sklearn.preprocessing import StandardScaler
import numpy as np

# Example data
data = np.array([[200], [300], [400], [600], [1000]])
# Initialize the scaler
scaler = StandardScaler()
# Fit and transform the data
scaled_data = scaler.fit_transform(data)
print("Original data:\n", data)
print("Scaled data:\n", scaled_data)
```

Original data:

```
[[ 200]
 [ 300]
 [ 400]
 [ 600]
[1000]]
```

Scaled data:

```
[[-1.06066017]
 [-0.70710678]
 [-0.35355339]
 [ 0.35355339]
 [ 1.76776695]]
```


Normalization

- **Normalization by decimal scaling** normalizes by moving the decimal point of values of attribute A.
- The number of decimal points moved depends on the maximum absolute value of A.
- A value, $\mathbf{v_i}$, of **A** is normalized to $\mathbf{v_i'}$ by computing
$$v'_i = \frac{v_i}{10^j}$$
where j is the smallest integer such that $\max(|\mathbf{v_i'}|) < 1$
- Suppose that the recorded values of A range from **−986 to 917**. The maximum absolute value of **A** is **986**. To normalize by decimal scaling, we therefore divide each value by **1000 (i.e., $j = 3$)** so that **−986** normalizes to **−0.986** and **917** normalizes to **0.917**.

```
import numpy as np

# Example data
data = np.array([200, 300, 400, 600, 1000])

# Find j
max_val = np.max(np.abs(data))
j = len(str(int(max_val)))

# Apply decimal scaling
scaled_data = data / (10 ** j)

print("Original data:", data)
print("Scaled data:", scaled_data)
```

Original data: [200 300 400 600 1000]

Scaled data: [0.02 0.03 0.04 0.06 0.1]

Feature Encoding

- Feature encoding converts categorical data into a numeric format so that machine learning algorithms can process it.
- Types of Categorical Features
 - Nominal: Categories with no inherent order.
 - Examples: color, city, water source type
 - Ordinal: Categories with a defined order.
 - Examples: water quality classes (poor < fair < good)

Feature Encoding – Label Encoding

- Assigns each category a unique integer.
- Pros:
 - Simple and fast
 - Works for ordinal data
- Cons:
 - Imposes an arbitrary order for nominal data
 - Not suitable for tree-based models if nominal

Feature Encoding – Label Encoding

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()  
cities = ['Kathmandu', 'Pokhara', 'Biratnagar']  
encoded = le.fit_transform(cities)  
print(encoded)
```

✓ 0.7s

[1 2 0]

When to use:

Ordinal categories

Small number of categories

Feature Encoding – One Hot Encoding

- Creates binary columns for each category.
- Pros:
 - No ordinal assumption
 - Works well for nominal data
- Cons:
 - Increases dimensionality
 - May slow down algorithms if many categories

Feature Encoding – One Hot Encoding

```
import pandas as pd
```

```
df = pd.DataFrame({'source': ['river', 'lake', 'well']})  
df_encoded = pd.get_dummies(df, columns=['source'])  
print(df_encoded)
```

✓ 0.4s

	source_lake	source_river	source_well
0	False	True	False
1	True	False	False
2	False	False	True

Feature Encoding – Ordinal Encoding

- Converts categories to integers, preserving order.
- When to use:
 - When categories have a meaningful order
 - Can be combined with scaling if needed

Feature Encoding – Ordinal Encoding

```
from sklearn.preprocessing import OrdinalEncoder
```

```
encoder = OrdinalEncoder(categories=[['poor', 'fair', 'good']])  
quality = [['poor'], ['good'], ['fair']]  
encoded = encoder.fit_transform(quality)  
print(encoded)
```

✓ 0.0s

```
[[0.]  
 [2.]  
 [1.]]
```

Contents

3.1 Advanced Pandas operations: Merging, joining, reshaping, pivoting

3.2 Handling missing, categorical, and time-series data

3.3 Feature transformation, scaling, and encoding

3.4 Memory optimization and efficient data processing

3.5 Building a reusable data-cleaning pipeline

3.6 Introduction to data pipeline components

Memory Optimization & Efficient Data Processing

- Large datasets and complex computations can exhaust memory.
- Efficient processing reduces runtime and system crashes.
- Critical for big data, ML models, and real-time applications.

Memory Optimization & Efficient Data Processing

- Common Memory Issues
 - Loading entire datasets into memory.
 - Unnecessary data copies.
 - Inefficient data types (e.g., float64 instead of float32).
 - Memory leaks in long-running processes.

Strategies for Memory Optimization

a) Data Types

- Use smaller numeric types: int8, float32.
- Convert categorical columns to category in pandas.

b) Chunking & Streaming

- Process data in batches instead of loading all at once.
- Example: `pd.read_csv('file.csv', chunksize=10000)`

Strategies for Memory Optimization

c) In-place Operations

- Modify data structures without creating copies.
- Example: `df.drop(columns=['col'], inplace=True)`

d) Garbage Collection

- Free unused objects manually: `import gc; gc.collect()`

e) Sparse Representations

- Use sparse matrices for mostly-zero data.
- Example: `scipy.sparse.csr_matrix()`

Efficient Data Processing

a) Vectorization

- Replace loops with vectorized operations (NumPy, pandas).
- Example: `df['sum'] = df['a'] + df['b']`

b) Parallel Processing

- Use multiprocessing or joblib for CPU-bound tasks.
- Example: `from multiprocessing import Pool`

c) Memory-Mapped Files

- Access large files on disk without loading all into memory.
- Example: `np.memmap('largefile.dat', dtype='float32')`

d) Lazy Evaluation

- Libraries like Dask, Vaex, or PySpark compute only when needed

Contents

3.1 Advanced Pandas operations: Merging, joining, reshaping, pivoting

3.2 Handling missing, categorical, and time-series data

3.3 Feature transformation, scaling, and encoding

3.4 Memory optimization and efficient data processing

3.5 Building a reusable data-cleaning pipeline

3.6 Introduction to data pipeline components

Data-Cleaning Pipeline

- Messy data might be missing values, duplicates, inconsistent formats, outliers.
- Motivation for a reusable pipeline: consistency, efficiency, scalability.

Goals of the Data-Cleaning Pipeline

- Automatically handle common data issues:
 - Missing values
 - Duplicates
 - Incorrect data types
 - Outliers
 - Categorical inconsistencies
- Modular and reusable across datasets.
- Easily extendable for new cleaning rules.

Data-Cleaning Pipeline Components

Step 1: Data Loading & Initial Inspection

Step 2: Missing Data Handling

Step 3: Duplicates Removal

Step 4: Data Type Correction

Step 5: Outlier Detection & Handling

Step 6: Feature Transformation (categorical encoding, scaling)

Step 7: Export Cleaned Data

Handling Missing Data

- Strategies:
 - Drop rows/columns
 - Fill with mean/median/mode
 - Advanced: KNN or interpolation
- Importance of logging what was filled/dropped for reproducibility.

Duplicate Handling

- Detect duplicates (`.duplicated()`)
- Remove duplicates (`.drop_duplicates()`)
- Maintain first occurrence or aggregate if needed.

Data Type Correction

- Convert columns to proper types:
 - Numeric, categorical, datetime
- Reduces errors in ML models and calculations.

Outlier Detection

- Methods:
 - Z-score
 - IQR (Interquartile Range)
 - Domain-specific thresholds
- Options: remove, clip, or transform.

Feature Transformation

- Scaling numeric features (StandardScaler, MinMaxScaler)
 - Encoding categorical features (OneHotEncoder, LabelEncoder)
- Pipeline integration for ML readiness.

Implementation

Contents

- 3.1 Advanced Pandas operations: Merging, joining, reshaping, pivoting
- 3.2 Handling missing, categorical, and time-series data
- 3.3 Feature transformation, scaling, and encoding
- 3.4 Memory optimization and efficient data processing
- 3.5 Building a reusable data-cleaning pipeline
- 3.6 Introduction to data pipeline components**

Data Pipeline

- A data pipeline is a set of automated steps to move, transform, and prepare data for analysis or machine learning.
- Key goals:
 - Ensure data quality
 - Enable scalable processing
 - Reduce manual effort

Data Pipeline Components

- Data Ingestion
 - Collect data from multiple sources.
- Data Validation
 - Check for errors, missing values, or inconsistencies.
- Data Cleaning & Transformation
 - Standardize formats, encode categorical variables, handle missing values.
- Data Storage
 - Save processed data in databases, data lakes, or files.
- Data Analysis/Modeling
 - Use data for analytics or ML tasks.
- Monitoring & Logging
 - Track pipeline performance and detect errors.

Data Ingestion

- Sources: CSV, Excel, databases, APIs, streaming data
- Tools: Python libraries (pandas, requests), ETL tools (Airflow, Prefect)
- Best practices:
 - Automate extraction
 - Handle large datasets efficiently

Data Validation

- Check for:
 - Missing or null values
 - Incorrect data types
 - Outliers and inconsistencies
- Techniques: pandas functions (isnull(), dtypes)

Data Cleaning & Transformation

- Cleaning
 - Remove duplicates, fill or drop missing values
- Transformation:
 - Scaling and normalization
 - Encoding categorical variables
 - Aggregation and pivoting
- Goal: Make data ready for analysis or ML

Data Storage

- Options:
 - Relational databases (MySQL, PostgreSQL)
 - NoSQL databases (MongoDB)
 - Files (CSV, JSON)
- Consider:
 - Read/write speed
 - Compression and storage efficiency

Data Analysis/Modeling

- Output of the pipeline feeds into:
 - Descriptive analytics (charts, summary statistics)
 - Predictive modeling (ML models)
 - Reporting dashboards

Monitoring & Logging

- Monitor for pipeline failures, data drift, and processing delays
- Tools: Airflow logs, custom logging in Python
- Enables early detection of issues and ensures pipeline reliability

Data Pipeline Components

From Raw Data to Insights

